EXERCÍCIOS - CORROTINAS ASYNCIO

Objetivo: Analisar, compreender e construir soluções assíncronas com a lib Asyncio
(Obs.: Não utilize outras formas de concorrência além da asyncio)

1. É possível iniciar a execução de uma função assíncrona através da *keyword* **`await`**, sem que a mesma tenha sido criada antes via **`asyncio.create_task()`**? Experimente! Qual a diferença entre ambas?

2. O que acontece na execução de um programa assíncrono (*asyncio*) se uma determinada função **`main()`** tiver sido finalizada/concluída antes de outras tarefas assíncronas lançadas por ela?

3. Uma função **`async`** pode receber valores via parâmetros? Como executar uma *task* assíncrona enviando dados por parâmetros?

4. Uma função **`async`** pode retornar valor como resultado? Como obter o valor retornado?

5. É possível cancelar a execução de uma função assíncrona após iniciada? Como isso é feito?

6. É possível que duas ou mais *tasks* assíncronas compartilhem uma mesma estrutura de dados para leitura e escrita (p.ex. Uma mesma lista enviada por parâmetros)? Faça um exemplo para provar.

7. Lembra do problema inicial da disciplina? Crie uma função **`async def print(lista)`** que imprime em loop infinito, e a cada 1 segundo, o conteúdo da lista recebida. Outra função **`async def main()`** deve criar/iniciar a task **`print`** e continuar ouvindo por novos elementos para ingresso na lista. Quando **`main()`** ouvir o valor '0' (zero) o programa deve ser encerrado imediatamente.
Atenção! O método tradicional de entrada no python (input) é síncrono, e como já vimos, isso é um baita problema para funções assíncronas! Tente solucionar esse problema usando o método padrão `input()` e depois compare com o método `ainput()` da lib `aioconsole`

8. Pesquise sobre os principais métodos da lib **`asyncio`**. Veja as diferenças e aplicabilidades de cada.

```
asyncio.gather()
async with asyncio.TaskGroup() as tg
asyncio.wait()
asyncio.wait_for()
```

9. Considerando a seguinte função assíncrona…

```
async def task(id, segs):
    await asyncio.sleep(segs)
    print(f'Task {id} finalizada em {segs} segundos.')
    return f'{id} ok'
```

a) Gere uma lista (use *List Comprehension*) de 10 chamadas **`task()`**, e invoque-as pelo método **`gather()`**
b) Como obter os resultados (retorno) das 10 chamadas assíncronas agrupadas pelo gather? Pesquise e imprima o retorno. O que você percebe de interessante?
c) Obtenha o mesmo resultado anterior, mas através do método **`asyncio.TaskGroup()`**
d) Compare ambas estruturas de agrupamento de tarefas, e suas diferenças - observe a cláusula **`await`**
e) Crie uma nova versão, mas que agora encerra a execução quando a primeira **`task()`** finalizar.
f) Crie uma nova versão, mas com **`timeout`** de 3 segundos.

10. Pesquise a diferença entre as estruturas **`async with`** e **`async for`**. Através de qual é possível criar uma função geradora assíncrona? Faça um exemplo real.

LABORATÓRIO PRÁTICO

Parte 1 :

Implemente um programa assíncrono que lance 10 *tasks* produtoras de conteúdo, sob seguinte assinatura:

```
async def producer(id, fila: asyncio.Queue)
```

Tasks produtoras devem alimentar colaborativamente a mesma fila (utilize objeto asyncio.Queue) com valores **ordenados** de 1 até 30.

O intervalo entre a produção novos números deve possuir latência simulada de 1 a 3 segundos. A geração do novo número deve observar a quantidade atual de números na fila, e com base neste, adicionar o próximo valor.

P.ex.:

```
v = fila.qsize()
await fila.put(v+1)
```

A cada novo número gerado, a função deve imprimir: `print(f'Produtor {id} produziu {v+1}')`

Parte 2 :

Continuando o problema anterior, lance 5 *tasks* consumidoras de conteúdo, sob seguinte assinatura:

```
async def consumer(id, fila: asyncio.Queue)
```

As *Tasks* consumidoras deverão obter valores da mesma fila em formato FIFO e dormir (*sleep*) pelo tempo correspondente à metade do valor obtido (simulando o tempo de processamento do evento obtido).

```
v = await fila.get()
await asyncio.sleep(v/2)
```

A cada novo número consumido, a função deve imprimir: `print(f'\t\t\t\tConsumidor {id} consumiu {v}')`

Análises Complementares do Laboratório…

1) ***Observe que quando acrescentamos tarefas consumidoras, as tasks produtoras começam a produzir conteúdos repetitivos, já que uma vez que a fila está sendo consumida concorrentemente o seu tamanho se torna variável…***

2) ***Altere o comportamento dos produtores, fazendo com que o valor produzido considere o último valor da fila (e não a quantidade de elementos da fila). Utilize:*** `v = fila._queue[-1]`

3) ***O que aconteceria quando há mais consumidores do que os produtores? Inverta a quantidade de produtores e consumidores, teste o cenário e avalie o comportamento da solução.***

4) ***Perceba que neste problema todo "evento" deve ser único e consumido apenas uma vez. Como seria possível garantir este requisito? Quais destas soluções seriam viáveis? Implemente e analise…***
   (a) Função Geradora Assíncrona (para abastecer as tasks produtoras)
   (b) Nova Fila Limitada (com último valor produzido para auxiliar as tasks produtoras)

5) ***Perceba também que a execução do algoritmo (tasks produtoras e consumidoras) devem ser finitas. Como poderíamos estabelecer um ponto de parada para todas as tarefas, e consequentemente, o fim da execução do programa?***